LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 1”

Disusun Oleh:
Zaki Misucha Andefi
2511533006
Dosen Pengampu **:** Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum pekan 1 mata kuliah Struktur Data ini dapat diselesaikan dengan baik.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna, oleh karena itu saran dan kritik sangat diharapkan demi perbaikan ke depannya. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun para pembaca.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Langkah Kerja 2
2.2 Program Class Jam 2
2.3 Program Driver Sederhana 3
2.4 Program Driver Input 3
2.5 Program Perulangan for 4 ..... **Error! Bookmark not defined.**
2.6 Program Nested For 0 ..... **Error! Bookmark not defined.**
2.7 Program Nested For 1 ..... **Error! Bookmark not defined.**
2.8 Program Nested for 2 ..... **Error! Bookmark not defined.**
BAB III KESIMPULAN 4
DAFTAR PUSTAKA 5

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada perkembangan teknologi saat ini, pemrograman menjadi salah satu keterampilan dasar yang sangat penting, khususnya dalam bidang informatika. Salah satu konsep utama dalam pemrograman adalah bagaimana cara merepresentasikan dan mengelola data secara efektif dan terstruktur. Oleh karena itu, pemahaman mengenai struktur data dan pemrograman berorientasi objek (Object-Oriented Programming/OOP) menjadi hal yang sangat diperlukan bagi mahasiswa.

Dalam praktikum ini, mahasiswa diperkenalkan pada implementasi konsep struktur data sederhana melalui pembuatan class dalam bahasa pemrograman Java. Class yang dibuat merepresentasikan objek waktu, yaitu jam, menit, dan detik. Dengan menggunakan class *Jam*, mahasiswa dapat memahami bagaimana suatu data tidak hanya disimpan, tetapi juga dapat diolah menggunakan berbagai operasi seperti konversi, perbandingan, serta operasi aritmatika.

Selain itu, penggunaan driver program juga memberikan gambaran nyata tentang bagaimana suatu class digunakan dalam program utama. Mahasiswa dapat melihat bagaimana objek dibuat, dimanipulasi, serta bagaimana interaksi antara user dengan program melalui proses input dan output. Hal ini membantu meningkatkan pemahaman terhadap alur eksekusi program secara keseluruhan.

Melalui praktikum ini, diharapkan mahasiswa tidak hanya memahami konsep dasar pemrograman, tetapi juga mampu mengaplikasikan konsep tersebut dalam bentuk program yang terstruktur, sistematis, dan mudah dipahami. Dengan demikian, praktikum ini menjadi langkah awal yang penting dalam mempelajari materi struktur data yang lebih kompleks di tahap selanjutnya.

1.2 Tujuan

- Memahami konsep class dan object.
- Mengimplementasikan operasi pada objek.
- Memahami manipulasi data waktu

1.3 Manfaat

Mahasiswa mampu memahami konsep OOP dan struktur data sederhana.

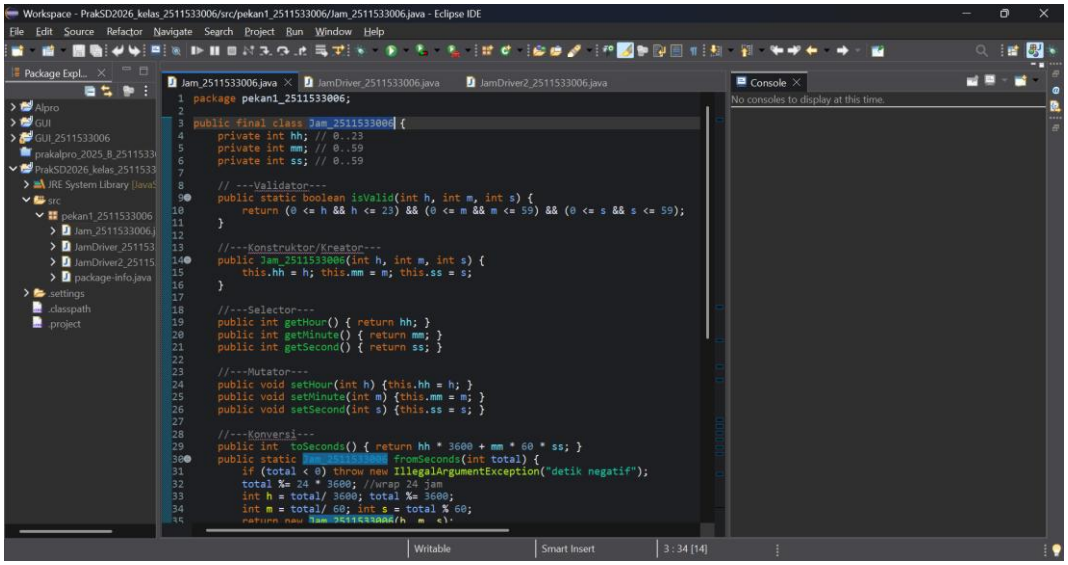
BAB II

PEMBAHASAN

2.1 Langkah Kerja

1. Membuka IDE (Eclipse).
2. Membuat package baru bernama pekan1_2511533006.
3. Menambahkan beberapa class:
   - Jam_2511533006, JamDriver_2511533006, JamDriver2_2511533006
4. Menjalankan program satu per satu dan mengamati hasilnya.
5. Menuliskan analisis output dari masing-masing program.

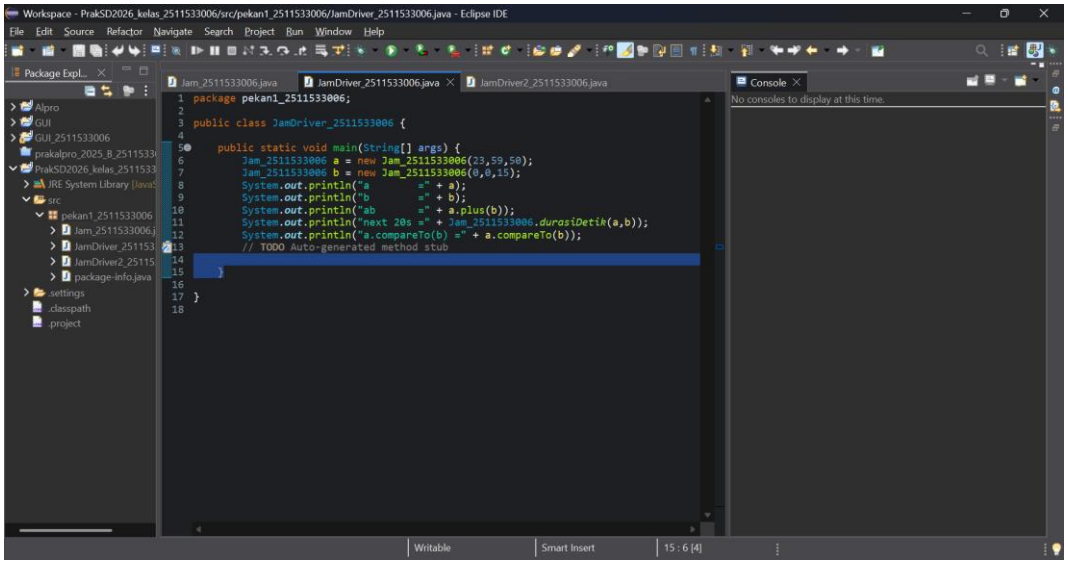
2.2 Program Class Jam



Analisis Hasil:

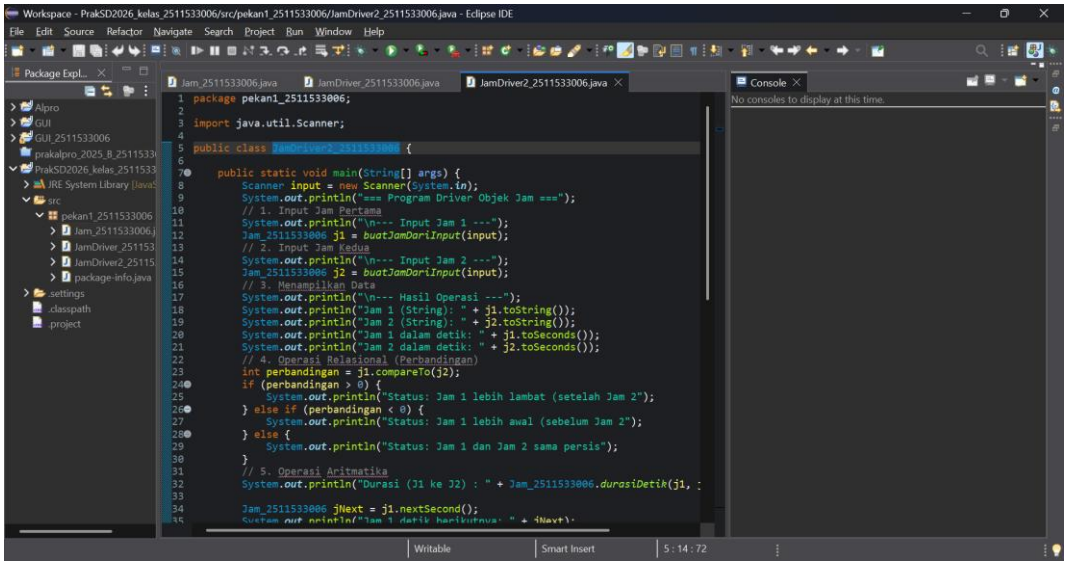
Class Jam digunakan untuk menyimpan dan mengolah data waktu dengan berbagai operasi seperti konversi dan aritmatika.

2.3 Program Driver Sederhana



Analisis: Driver ini digunakan untuk menguji fungsi class Jam dengan nilai langsung.

2.4 Program Driver Input



Analisis: Program ini memungkinkan input user dan melakukan validasi sebelum diproses.

BAB III
KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan class dalam pemrograman Java sangat efektif untuk merepresentasikan suatu entitas, dalam hal ini adalah waktu (jam, menit, dan detik). Dengan adanya class *Jam*, data waktu tidak hanya dapat disimpan, tetapi juga dapat dimanipulasi melalui berbagai operasi seperti konversi ke detik, perbandingan waktu, serta operasi aritmatika seperti penjumlahan dan pengurangan waktu.

Selain itu, melalui pembuatan driver program, baik yang menggunakan nilai langsung maupun input dari pengguna, mahasiswa dapat memahami bagaimana objek digunakan dalam program secara nyata. Driver sederhana membantu dalam pengujian fungsi, sedangkan driver berbasis input memberikan gambaran interaksi antara user dengan program serta pentingnya validasi data sebelum diproses.

Praktikum ini juga memperkuat pemahaman mengenai konsep Object-Oriented Programming (OOP), seperti enkapsulasi melalui penggunaan atribut private dan method public, serta penggunaan method sebagai operasi terhadap data. Dengan membagi program ke dalam beberapa method, kode menjadi lebih terstruktur, mudah dibaca, dan dapat digunakan kembali.

Secara keseluruhan, praktikum ini memberikan dasar yang kuat dalam memahami struktur data sederhana berbasis objek, serta melatih kemampuan berpikir logis dan sistematis dalam menyelesaikan permasalahan menggunakan bahasa pemrograman Java. Pengetahuan ini akan sangat berguna untuk materi selanjutnya yang lebih kompleks dalam bidang struktur data dan pemrograman.

DAFTAR PUSTAKA

[1] H. M. Deitel dan P. J. Deitel, *Java: How to Program*, 11th ed. New Jersey, NJ: Pearson, 2017.

Tugas Struktur Data

1. Kelas ADT: Mobil.java

```java
package pekan1_2511533006;

//Nama: Zaki Misucha Andefi - NIM: 2511533006

public class Mobil {
 // 1. Atribut
 private String nama;
 private int tahun;
 private int cc;
 private long harga;
 private String merk;

 // Constructor untuk memudahkan pembuatan objek
 public Mobil(String nama, int tahun, int cc, long harga, String merk) {
     this.nama = nama;
     this.tahun = tahun;
     this.cc = cc;
     this.harga = harga;
     this.merk = merk;
 }

 // 2. Selektor (Getter) - Mengambil nilai atribut
 public String getNama() { return nama; }
 public int getTahun() { return tahun; }
 public int getCc() { return cc; }
 public long getHarga() { return harga; }
 public String getMerk() { return merk; }

 // 3. Mutator (Setter) - Mengubah nilai atribut
 public void setNama(String nama) { this.nama = nama; }
 public void setTahun(int tahun) { this.tahun = tahun; }
 public void setCc(int cc) { this.cc = cc; }
 public void setHarga(long harga) { this.harga = harga; }
 public void setMerk(String merk) { this.merk = merk; }

 // Method tambahan untuk menampilkan data mobil
 public void tampilkanInfo() {
     System.out.println(merk + " " + nama + " (" + tahun + ") | " + cc + "cc | Rp" + harga);
 }
}
```

Analisis :

- Penerapan Enkapsulasi (Information Hiding): Atribut seperti nama, tahun, dan harga dideklarasikan dengan kata kunci private. Ini adalah praktik standar ADT. Tujuannya adalah melindungi data agar tidak bisa diubah secara sembarangan dari luar kelas. Data hanya bisa diakses atau diubah melalui "pintu" resmi yang telah disediakan.
- Peran Konstruktor: Method Mobil(...) yang memiliki nama sama dengan kelasnya disebut konstruktor. Fungsinya adalah menyiapkan "state awal" saat memori

untuk satu objek mobil baru dialokasikan. Konstruktor ini memaksa kita untuk langsung mengisi kelima atribut mobil (nama, tahun, cc, harga, merk) saat objek dibuat.

- Selektor (Getter): Metode seperti getNama() atau getHarga() adalah "pintu keluar" data. Jika kelas lain butuh membaca nilai atribut yang di- *private*, mereka harus memanggil method ini.
- Mutator (Setter): Metode seperti setNama(...) adalah "pintu masuk" data. Jika ada kebutuhan untuk merevisi data mobil (misalnya harganya naik), perubahan harus melewati method ini. Nantinya, di dalam setter, kamu bisa menambahkan validasi (misal: harga tidak boleh minus) sebelum data benar-benar diubah.

2. Kelas Driver: DriverMobil.java

```
package pekan1_2511533006;

//Nama: Zaki Misucha Andefi - NIM: 2511533006

public class DriverMobil {

 // Inisialisasi Array biasa dengan kapasitas maksimal
 private static Mobil[] dataMobil = new Mobil[100];
 // Variabel untuk melacak jumlah mobil yang ada di dalam array
 private static int jumlahMobil = 0;

 public static void main(String[] args) {
     System.out.println("=== PROGRAM ADT MOBIL ===");
     System.out.println("NIM: 3006\n");

     // Menambahkan beberapa mobil
     tambahMobil(new Mobil("Avanza", 2020, 1300, 150000000, "Toyota"));
     tambahMobil(new Mobil("Voxy", 2025, 2000, 600000000, "Toyota"));
     tambahMobil(new Mobil("Alphard Hybrid", 2025, 2500, 1400000000,
"Toyota"));

     tampilSemuaMobil();

     // Menguji penghapusan mobil
     System.out.println("\nMenghapus mobil Voxy dari array...");
     hapusMobil("Voxy");

     tampilSemuaMobil();
 }

 // --- Method Tambah Mobil ---
 public static void tambahMobil(Mobil mobilBaru) {
     if (jumlahMobil < dataMobil.length) {
         dataMobil[jumlahMobil] = mobilBaru; // Masukkan ke index yang
kosong
         jumlahMobil++;                      // Tambah counter
         System.out.println("[+] Berhasil menambahkan: " +
mobilBaru.getNama());
     } else {
         System.out.println("[-] Gagal: Kapasitas array sudah penuh!");
```

```java
    }
}

// --- Method Hapus Mobil (Berdasarkan Nama) ---
public static void hapusMobil(String namaTarget) {
    int indexDihapus = -1;

    // 1. Cari index array dari mobil yang ingin dihapus
    for (int i = 0; i < jumlahMobil; i++) {
        if (dataMobil[i].getNama().equalsIgnoreCase(namaTarget)) {
            indexDihapus = i;
            break;
        }
    }

    if (indexDihapus != -1) {
        for (int i = indexDihapus; i < jumlahMobil - 1; i++) {
            dataMobil[i] = dataMobil[i + 1];
        }
        dataMobil[jumlahMobil - 1] = null;
        jumlahMobil--;

        System.out.println("[+] Mobil " + namaTarget + " berhasil
dihapus.");
    } else {
        System.out.println("[-] Mobil " + namaTarget + " tidak
ditemukan.");
    }
}

// --- Method Bantuan untuk Menampilkan Array ---
public static void tampilSemuaMobil() {
    System.out.println("\nDaftar Mobil Saat Ini:");
    if (jumlahMobil == 0) {
        System.out.println("Data masih kosong.");
    } else {
        for (int i = 0; i < jumlahMobil; i++) {
            System.out.print((i + 1) + ". ");
            dataMobil[i].tampilkanInfo();
        }
    }
}
}
```

Analisis :

- Penyimpanan Data Berbasis Array Statis:

Program menggunakan Mobil[] dataMobil = new Mobil[100];. Artinya, program memesan ruang di memori utama (RAM) secara berurutan (*contiguous*) untuk menampung maksimal 100 objek mobil. Kelemahannya: jika kita butuh menyimpan 101 mobil, program akan *error* (*Array Index Out Of Bounds*).

- Fungsi Krusial Variabel jumlahMobil:

bertindak sebagai Pointer (Penunjuk Indeks).

- Karena array dimulai dari indeks 0, jika ada 3 mobil di dalam array, indeks yang terisi adalah 0, 1, dan 2.
- Nilai jumlahMobil saat itu adalah 3.
- Secara kebetulan (dan matematis), angka 3 ini adalah indeks kosong pertama yang siap diisi jika ada penambahan mobil baru. Inilah mengapa pada method tambahMobil kita langsung menggunakan dataMobil[jumlahMobil] = mobilBaru.

- Algoritma Penambahan Data ($O(1)$ - Konstan):

Proses penambahan data sangat cepat. Program tidak perlu mencari tempat kosong satu per satu. Selama jumlahMobil belum menyentuh angka 100, program langsung menaruh objek baru tepat di indeks jumlahMobil, lalu menaikkan angka jumlahMobil sebesar 1.

- Algoritma Penghapusan Data ($O(N)$ - Linear):

1. Pencarian (Linear Search): Program harus membaca array dari indeks 0 sampai ujung untuk mencari nama mobil yang cocok.
2. Pergeseran (Shifting): Jika mobil yang dihapus ada di tengah (misal indeks ke-1), kita tidak bisa sekadar membiarkan indeks ke-1 itu kosong (berisi null). Jika dibiarkan kosong, struktur array akan berlubang dan menyulitkan pembacaan data. Oleh karena itu, semua elemen yang ada di sebelah kanannya (indeks ke-2, ke-3, dst) harus disalin dan digeser satu langkah ke kiri untuk menutup lubang tersebut.
3. Pembersihan: Setelah semua digeser, elemen paling kanan yang sebelumnya sudah digeser ke kiri, harus dihapus secara manual (diberi nilai null) agar tidak ada data ganda, barulah variabel jumlahMobil dikurangi 1.